



深圳大学
Shenzhen University

互联网编程

第9章

服务器Socket

深圳大学计算机与软件学院

主要内容

- 使用ServerSocket
- 日志
- 构造服务器Socket
- 获得服务器Socket的有关信息
- Socket选项
- HTTP服务器



使用ServerSocket

- <http://docs.oracle.com/javase/8/docs/api/java/net/ServerSocket.html>
- ServerSocket类包含了使用Java编写服务器所需的全部内容，服务器程序的基本生命周期：
 - 构造ServerSocket（绑定某个端口）
 - accept方法监听（阻塞）并返回一个连接客户端的Socket
 - 调用Socket的getInputStream或getOutputStream方法获得IO流
 - 根据已协商的协议交互
 - 关闭连接
- 一个例子

```
try(ServerSocket server=new ServerSocket(PORT)){
    while(true){
        try (Socket connection=server.accept()){
            Writer out=new OutputStreamWriter(connection.getOutputStream());
            Date now=new Date();
            out.write(now.toString()+"\r\n");
            out.flush();
            connection.close();
        }catch (IOException ex){ }
    }
}catch(IOException ex){...}
```



使用ServerSocket

- 提供二进制数据
 - `OutputStream out=connection.getOutputStream();`
 - `byte[] time=new byte[4]; .....`
 - `out.write(time) .....`

Ex9-2 TimeServer

时间服务器例子: 一个遵循RFC868时间协议的迭代时间服务器。当客户端连接时, 服务器发送一个4字节大端无符号整数, 指出从GMT1900.1.1 12:00A.M.后经过的秒数。

结合例子 时间协议客户端 ex8-3 Time

- 多线程服务器
 - 待处理连接队列
 - 队列长度有限
 - 进程、线程
 - 避免异常阻塞

Ex9-3 MultithreadDaytimeServer

多线程时间服务器例子

- 线程池
 - 提高性能, 避免崩溃
 - ExecutorService
 - Callable<void>

Ex9-4 PooledDaytimeServer

使用线程池的时间服务器例子

你使用socket编写的服务器端程序是如何编写的? 存在被攻击的可能吗? 考虑并发大量连接的问题吗?



```
1 import java.io.*;
2 import java.net.*;
3 import java.util.Date;
4 public class TimeServer {
5     public final static int PORT = 37;
6     public static void main(String[] args) {
7         // The time protocol sets the epoch at 1900,
8         // the Date class at 1970. This number
9         // converts between them.
10        long differenceBetweenEpochs = 2208988800L;
11        try (ServerSocket server = new ServerSocket(PORT)) {
12            while (true) {
13                try (Socket connection = server.accept()) {
14                    OutputStream out = connection.getOutputStream();
15                    Date now = new Date();
16                    long msSince1970 = now.getTime();
17                    long secondsSince1970 = msSince1970/1000;
18                    long secondsSince1900 = secondsSince1970
19                        + differenceBetweenEpochs;
20                    byte[] time = new byte[4];
21                    time[0]
22                        = (byte) ((secondsSince1900 & 0x00000000FF000000L) >> 24);
23                    time[1]
24                        = (byte) ((secondsSince1900 & 0x0000000000FF0000L) >> 16);
25                    time[2]
26                        = (byte) ((secondsSince1900 & 0x000000000000FF00L) >> 8);
27                    time[3] = (byte) (secondsSince1900 & 0x00000000000000FFL);
```

Ex9-2 TimeServer

时间服务器例子: 一个遵循RFC868时间协议的迭代时间服务器。当客户端连接时, 服务器发送一个4字节大端无符号整数, 指出从GMT1900.1.1 12:00A.M.后经过的秒数。

结合例子 时间协议客户端 ex8-3 Time

TimeServer.java

```
28         out.write(time);
29         out.flush();
30     } catch (IOException ex) {
31         System.err.println(ex.getMessage());
32     }
33 }
34 } catch (IOException ex) {
35     System.err.println(ex);
36 }
37 }
38 }
```

```

1 import java.net.*;
2 import java.io.*;
3 import java.util.Date;
4 public class MultithreadedDaytimeServer {
5     public final static int PORT = 13;
6     public static void main(String[] args) {
7         try (ServerSocket server = new ServerSocket(PORT)) {
8             while (true) {
9                 try {
10                     Socket connection = server.accept();
11                     Thread task = new DaytimeThread(connection);
12                     task.start();
13                 } catch (IOException ex) {}
14             }
15         } catch (IOException ex) {
16             System.err.println("Couldn't start server");
17         }
18     }
19
20     private static class DaytimeThread extends Thread {
21         private Socket connection;
22         DaytimeThread(Socket connection) {
23             this.connection = connection;
24         }
25         @Override

```

Ex9-3 MultithreadDaytimeServer

多线程时间服务器例子

MultithreadedDaytimeServer.java

```
26 public void run() {
27     try {
28         Writer out = new OutputStreamWriter(connection.getOutputStream());
29         Date now = new Date();
30         out.write(now.toString() + "\r\n");
31         out.flush();
32     } catch (IOException ex) {
33         System.err.println(ex);
34     } finally {
35         try {
36             connection.close();
37         } catch (IOException e) {
38             // ignore;
39         }
40     }
41 }
42 }
43 }
```

Ex9-4 PooledDaytimeServer

使用线程池的时间服务器例子

PooledDaytimeServer.java

```
1 import java.io.*;
2 import java.net.*;
3 import java.util.*;
4 import java.util.concurrent.*;
5 public class PooledDaytimeServer {
6     public final static int PORT = 13;
7     public static void main(String[] args) {
8         ExecutorService pool = Executors.newFixedThreadPool(50);
9         try (ServerSocket server = new ServerSocket(PORT)) {
10             while (true) {
11                 try {
12                     Socket connection = server.accept();
13                     Callable<Void> task = new DaytimeTask(connection);
14                     pool.submit(task);
15                 } catch (IOException ex) {}
16             }
17         } catch (IOException ex) {
18             System.err.println("Couldn't start server");
19         }
20     }
21
22     private static class DaytimeTask implements Callable<Void> {
23
24         private Socket connection;
25         DaytimeTask(Socket connection) {
26             this.connection = connection;
27         }
28     }
29 }
```

Ex9-4 PooledDaytimeServer

使用线程池的时间服务器例子

PooledDaytimeServer.java

```
28 @Override
29 public void call() {
30     try {
31         Writer out = new OutputStreamWriter(connection.getOutputStream());
32         Date now = new Date();
33         out.write(now.toString() + "\r\n");
34         out.flush();
35     } catch (IOException ex) {
36         System.err.println(ex);
37     } finally {
38         try {
39             connection.close();
40         } catch (IOException e) {
41             // ignore;
42         }
43     }
44     return null;
45 }
46 }
47 }
```

使用ServerSocket

- 用Socket写入服务器

- echo服务器， **ex9-5 EchoServer**

- 需服务器端程序接受一个连接，同时获取该连接的输入流和输出流。

- 关闭服务器Socket

- 注意区分关闭Socket与关闭ServerSocket

- isClosed()与isBound()

- isOpen

- `ss.isBound && !ss.isClosed()`



```
1 import java.nio.*;
2 import java.nio.channels.*;
3 import java.net.*;
4 import java.util.*;
5 import java.io.IOException;
6 public class EchoServer {
7     public static int DEFAULT_PORT = 7;
8     public static void main(String[] args) {
9         int port;
10        try {
11            port = Integer.parseInt(args[0]);
12        } catch (RuntimeException ex) {
13            port = DEFAULT_PORT;
14        }
15        System.out.println("Listening for connections on port " + port);
16        ServerSocketChannel serverChannel;
17        Selector selector;
18        try {
19            serverChannel = ServerSocketChannel.open();
20            ServerSocket ss = serverChannel.socket();
21            InetSocketAddress address = new InetSocketAddress(port);
22            ss.bind(address);
23            serverChannel.configureBlocking(false);
24            selector = Selector.open();
25            serverChannel.register(selector, SelectionKey.OP_ACCEPT);
26        } catch (IOException ex) {
27            ex.printStackTrace();
28        }
```

```
28     return;
29 }
30
31 while (true) {
32     try {
33         selector.select();
34     } catch (IOException ex) {
35         ex.printStackTrace();
36         break;
37     }
38
39     Set<SelectionKey> readyKeys = selector.selectedKeys();
40     Iterator<SelectionKey> iterator = readyKeys.iterator();
41     while (iterator.hasNext()) {
42         SelectionKey key = iterator.next();
43         iterator.remove();
44         try {
45             if (key.isAcceptable()) {
46                 ServerSocketChannel server = (ServerSocketChannel) key.channel();
47                 SocketChannel client = server.accept();
48                 System.out.println("Accepted connection from " + client);
49                 client.configureBlocking(false);
50                 SelectionKey clientKey = client.register(
51                     selector, SelectionKey.OP_WRITE | SelectionKey.OP_READ);
52                 ByteBuffer buffer = ByteBuffer.allocate(100);
53                 clientKey.attach(buffer);
54             }
```

EchoServer

Echo服务器，用socket写入服务器，客户端负责关闭连接，服务器会生成最多500个线程

EchoServer.java

```
55         if (key.isReadable()) {
56             SocketChannel client = (SocketChannel) key.channel();
57             ByteBuffer output = (ByteBuffer) key.attachment();
58             client.read(output);
59         }
60         if (key.isWritable()) {
61             SocketChannel client = (SocketChannel) key.channel();
62             ByteBuffer output = (ByteBuffer) key.attachment();
63             output.flip();
64             client.write(output);
65             output.compact();
66         }
67     } catch (IOException ex) {
68         key.cancel();
69         try {
70             key.channel().close();
71         } catch (IOException cex) {}
72     }
73 }
74 }
75 }
76 }
```

日志

- 日志记录内容

- 请求
- 服务器错误：错误日志一般要重点关注
- 调试日志和生产日志

- 如何记录日志

- log4j
- <http://docs.oracle.com/javase/8/docs/api/java/util/logging/Logger.html>
- `private final static Logger logger=Logger.getLogger("requests");`
- `Logger.log(Level.SEVERE, "some msg" + ex. =getMessage(),ex)`
- Level: SEVERE\WARNING\INFO\CONFIG\FINE\FINER\FINEST
- `logger.info(new Date() + " ..." + someOtherUsefulMsg);`

LoggingDaytimeServer.java类实现记录请求和错误的daytime服务器,展示了如何为daytime服务器增加日志记录



LoggingDaytimeServer.java类实现记录请求和错误的daytime服务器,展示了如何为daytime服务器增加日志记录

LoggingDaytimeServer.java

```
1 import java.io.*;
2 import java.net.*;
3 import java.util.Date;
4 import java.util.concurrent.*;
5 import java.util.logging.*;
6 public class LoggingDaytimeServer {
7     public final static int PORT = 13;
8     private final static Logger auditLogger = Logger.getLogger("requests");
9     private final static Logger errorLogger = Logger.getLogger("errors");
10    public static void main(String[] args) {
11        ExecutorService pool = Executors.newFixedThreadPool(50);
12        try (ServerSocket server = new ServerSocket(PORT)) {
13            while (true) {
14                try {
15                    Socket connection = server.accept();
16                    Callable<Void> task = new DaytimeTask(connection);
17                    pool.submit(task);
18                } catch (IOException ex) {
19                    errorLogger.log(Level.SEVERE, "accept error", ex);
20                } catch (RuntimeException ex) {
21                    errorLogger.log(Level.SEVERE, "unexpected error " + ex.getMessage(), ex);
22                }
23            }
24        } catch (IOException ex) {
25            errorLogger.log(Level.SEVERE, "Couldn't start server", ex);
26        } catch (RuntimeException ex) {
27            errorLogger.log(Level.SEVERE, "Couldn't start server: " + ex.getMessage(), ex);
28        }
```

```

28     }
29 }
30 private static class DaytimeTask implements Callable<Void> {
31     private Socket connection;
32     DaytimeTask(Socket connection) {
33         this.connection = connection;
34     }
35     @Override
36     public Void call() {
37         try {
38             Date now = new Date();
39             // write the log entry first in case the client disconnects
40             auditLogger.info(now + " " + connection.getRemoteSocketAddress());
41             Writer out = new OutputStreamWriter(connection.getOutputStream());
42             out.write(now.toString() + "\r\n");
43             out.flush();
44         } catch (IOException ex) {
45             // client disconnected; ignore;
46         } finally {
47             try {
48                 connection.close();
49             } catch (IOException ex) {
50                 // ignore;
51             }
52         }
53         return null;
54     }

```

构造服务器Socket

■ 4个构造函数

- ❑ ServerSocket()
- ❑ ServerSocket(int port)
- ❑ ServerSocket(int port, int backlog)
- ❑ ServerSocket(int port, int backlog, InetAddress bindAddr)

■ 3个参数

- ❑ 监听端口，0表示自动分配
- ❑ 队列长度， $\geq$ 系统最大队列长度
- ❑ 绑定地址，多IP情况下的特定IP

■ 利用BindException查找本地端口

■ 构造但不绑定端口

- ❑ 用于在绑定前设置某些SO选项
- ❑ void bind(SocketAddress endpoint)
- ❑ void bind(SocketAddress endpoint, int backlog)
- ❑ SocketAddress可为null，表示待系统分配



LocalPortScanner.java 查找本地端口程序：尝试在各个端口创建ServerSocket对象，查看哪些端口失败，以此来检查本地机器的端口情况。

LocalPortScanner.java

```
1 import java.io.*;
2 import java.net.*;
3
4 public class LocalPortScanner {
5
6     public static void main(String[] args) {
7
8         for (int port = 1; port <= 65535; port++) {
9             try {
10                 // the next line will fail and drop into the catch block if
11                 // there is already a server running on the port
12                 ServerSocket server = new ServerSocket(port);
13             } catch (IOException ex) {
14                 System.out.println("There is a server on port " + port + ".");
15             }
16         }
17     }
18 }
```

获得服务器Socket的有关信息

- InetAddress getAddress()
- int getLocalPort()
- SocketAddress getLocalSocketAddress()

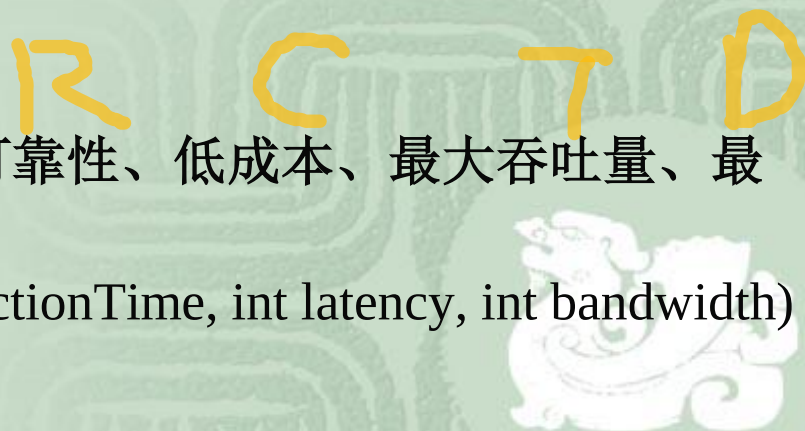
RandomPort.java 随机端口程序：ServerSocket构造函数允许为端口号传递0，
监听未指定的端口，利用此方法可以找出所监听的端口。

```
RandomPort.java
1 import java.io.*;
2 import java.net.*;
3 public class RandomPort {
4
5     public static void main(String[] args) {
6         try {
7             ServerSocket server = new ServerSocket(0);
8             System.out.println("This server runs on port "
9                 + server.getLocalPort());
10        } catch (IOException ex) {
11            System.err.println(ex);
12        }
13    }
14 }
```



Socket选项

- Socket选项指定了ServerSocket类所依赖的原生Socket如何发送和接收数据，支持3个选项：
- SO_TIMEOUT
 - 默认0表示永不超时
 - 自调用accept()后开始计时（阻塞等待时间）
- SO_REUSEADDR
 - 与客户端Socket的同名选项作用相同
- SO_RCVBUF
 - 设置的是哪个缓冲区的大小？
- 服务类型
 - TCP定义4个通用业务流类型：高可靠性、低成本、最大吞吐量、最小延迟
 - setPerformancePreferences(int connectionTime, int latency, int bandwidth)，数值越高越重要



HTTP服务器

- 简单HTTP服务器用途
 - 小且简单网站，定制服务器，节约资源
 - 仅处理一件事情，高效
 - 如一台专门服务器将常用文件存到RAM中
 - 专门的重定向服务器（也可用域名解析来解决这个问题）
- Java VS C/C++
 - HTTP服务器的瓶颈在带宽等I/O问题；
 - CPU计算能力一般不是瓶颈
 - Java优势：内存保护、垃圾自动回收、...



单文件HTTP服务器

- 无论什么请求，服务器始终发送同一个文件
- 过程：

- main:创建SingleFileHTTPServer对象

- 创建线程池

- 无限循环

- 有客户端连接

- 交由Handler对象处理

SingleFileHTTPServer.java



SingleFileHTTPServer.java提供同一个文件的HTTP服务器

SingleFileHTTPServer.java

```
1
2 import java.io.*;
3 import java.net.*;
4 import java.nio.charset.Charset;
5 import java.nio.file.*;
6 import java.util.concurrent.*;
7 import java.util.logging.*;
8 public class SingleFileHTTPServer {
9     private static final Logger logger = Logger.getLogger("SingleFileHTTPServer");
10    private final byte[] content;
11    private final byte[] header;
12    private final int port;
13    private final String encoding;
14    public SingleFileHTTPServer(String data, String encoding,
15        String mimeType, int port) throws UnsupportedOperationException {
16        this(data.getBytes(encoding), encoding, mimeType, port);
17    }
18    public SingleFileHTTPServer(
19        byte[] data, String encoding, String mimeType, int port) {
20        this.content = data;
21        this.port = port;
22        this.encoding = encoding;
23        String header = "HTTP/1.0 200 OK\r\n"
24            + "Server: OneFile 2.0\r\n"
25            + "Content-length: " + this.content.length + "\r\n"
26            + "Content-type: " + mimeType + "; charset=" + encoding + "\r\n\r\n";
27        this.header = header.getBytes(Charset.forName("US-ASCII"));
```

```
28     }
29     public void start() {
30         ExecutorService pool = Executors.newFixedThreadPool(100);
31         try (ServerSocket server = new ServerSocket(this.port)) {
32             logger.info("Accepting connections on port " + server.getLocalPort());
33             logger.info("Data to be sent:");
34             logger.info(new String(this.content, encoding));
35             while (true) {
36                 try {
37                     Socket connection = server.accept();
38                     pool.submit(new HTTPHandler(connection));
39                 } catch (IOException ex) {
40                     logger.log(Level.WARNING, "Exception accepting connection", ex);
41                 } catch (RuntimeException ex) {
42                     logger.log(Level.SEVERE, "Unexpected error", ex);
43                 }
44             }
45         } catch (IOException ex) {
46             logger.log(Level.SEVERE, "Could not start server", ex);
47         }
48     }
49     private class HTTPHandler implements Callable<Void> {
50         private final Socket connection;
51
52         HTTPHandler(Socket connection) {
53             this.connection = connection;
54         }
55     }
```

```
55 @Override
56 public void call() throws IOException {
57     try {
58         OutputStream out = new BufferedOutputStream(
59             connection.getOutputStream()
60         );
61         InputStream in = new BufferedInputStream(
62             connection.getInputStream()
63         );
64         // read the first line only; that's all we need
65         StringBuilder request = new StringBuilder(80);
66         while (true) {
67             int c = in.read();
68             if (c == '\r' || c == '\n' || c == -1) break;
69             request.append((char) c);
70         }
71         // If this is HTTP/1.0 or later send a MIME header
72         if (request.toString().indexOf("HTTP/") != -1) {
73             out.write(header);
74         }
75         out.write(content);
76         out.flush();
77     } catch (IOException ex) {
78         logger.log(Level.WARNING, "Error writing to client", ex);
79     } finally {
80         connection.close();
81     }
}
```

```
82     return null;
83 }
84 }
85 public static void main(String[] args) {
86     // set the port to listen on
87     int port;
88     try {
89         port = Integer.parseInt(args[1]);
90         if (port < 1 || port > 65535) port = 80;
91     } catch (RuntimeException ex) {
92         port = 80;
93     }
94     String encoding = "UTF-8";
95     if (args.length > 2) encoding = args[2];
96     try {
97         Path path = Paths.get(args[0]);
98         byte[] data = Files.readAllBytes(path);
99
100         String contentType = URLConnection.getFileNameMap().getContentTypeFor(args[0]);
101         SingleFileHTTPServer server = new SingleFileHTTPServer(data, encoding,
102             contentType, port);
103         server.start();
104     } catch (ArrayIndexOutOfBoundsException ex) {
105         System.out.println(
106             "Usage: java SingleFileHTTPServer filename port encoding");
107     } catch (IOException ex) {
108         logger.severe(ex.getMessage());
```

```
109     }
110 }
111 }
```

重定向HTTP服务器

- 将请求重定向到一个新网站

客户端: GET /nc/ HTTP/1.1

```
HTTP/1.0 302 FOUND
```

```
Date: Mon May 23 13:20:29 CST 2016
```

```
Server: Redirector 1.1
```

```
Location: http://www.szu.edu.cn/nc/
```

```
Content-type: text/html
```

```
<HTML><HEAD><TITLE>Document moved</TITLE></HEAD>
```

```
<BODY><H1>Document moved</H1>
```

```
The document /nc/ has moved to
```

```
<A HREF="http://www.szu.edu.cn/nc/">http://www.szu.edu.cn/nc/</A>.
```

```
Please update your bookmarks<P></BODY></HTML>
```

Redirector.java, HTTP重定向器, 一个简单有用的HTTP服务器



Redirector.java, HTTP重定向器, 一个简单有用的HTTP服务器

Redirector.java

```
1 import java.io.*;
2 import java.net.*;
3 import java.util.*;
4 import java.util.logging.*;
5 public class Redirector {
6     private static final Logger logger = Logger.getLogger("Redirector");
7     private final int port;
8     private final String newSite;
9     public Redirector(String newSite, int port) {
10         this.port = port;
11         this.newSite = newSite;
12     }
13     public void start() {
14         try (ServerSocket server = new ServerSocket(port)) {
15             logger.info("Redirecting connections on port "
16                 + server.getLocalPort() + " to " + newSite);
17             while (true) {
18                 try {
19                     Socket s = server.accept();
20                     Thread t = new RedirectThread(s);
21                     t.start();
22                 } catch (IOException ex) {
23                     logger.warning("Exception accepting connection");
24                 } catch (RuntimeException ex) {
25                     logger.log(Level.SEVERE, "Unexpected error", ex);
26                 }
27             }
28         }
29     }
30 }
```

```
28     } catch (BindException ex) {
29         logger.log(Level.SEVERE, "Could not start server.", ex);
30     } catch (IOException ex) {
31         logger.log(Level.SEVERE, "Error opening server socket", ex);
32     }
33 }
34 private class RedirectThread extends Thread {
35     private final Socket connection;
36     RedirectThread(Socket s) {
37         this.connection = s;
38     }
39     public void run() {
40         try {
41             Writer out = new BufferedWriter(
42                 new OutputStreamWriter(
43                     connection.getOutputStream(), "US-ASCII"
44                 )
45             );
46             Reader in = new InputStreamReader(
47                 new BufferedInputStream(
48                     connection.getInputStream()
49                 )
50             );
51             // read the first line only; that's all we need
52             StringBuilder request = new StringBuilder(80);
53             while (true) {
54                 int c = in.read();
```

```
55     if (c == '\r' || c == '\n' || c == -1) break;
56     request.append((char) c);
57 }
58 String get = request.toString();
59 String[] pieces = get.split("\\w*");
60 String theFile = pieces[1];
61 // If this is HTTP/1.0 or later send a MIME header
62 if (get.indexOf("HTTP") != -1) {
63     out.write("HTTP/1.0 302 FOUND\r\n");
64     Date now = new Date();
65     out.write("Date: " + now + "\r\n");
66     out.write("Server: Redirector 1.1\r\n");
67     out.write("Location: " + newSite + theFile + "\r\n");
68     out.write("Content-type: text/html\r\n\r\n");
69     out.flush();
70 }
71 // Not all browsers support redirection so we need to
72 // produce HTML that says where the document has moved to.
73 out.write("<HTML><HEAD><TITLE>Document moved</TITLE></HEAD>\r\n");
74 out.write("<BODY><H1>Document moved</H1>\r\n");
75 out.write("The document " + theFile
76     + " has moved to\r\n<A HREF=\"" + newSite + theFile + "\">"
77     + newSite + theFile
78     + "</A>.\r\n Please update your bookmarks<P>");
79 out.write("</BODY></HTML>\r\n");
80 out.flush();
81 logger.log(Level.INFO,
```

```
82         "Redirected " + connection.getRemoteSocketAddress());
83     } catch (IOException ex) {
84         logger.log(Level.WARNING,
85             "Error talking to " + connection.getRemoteSocketAddress(), ex);
86     } finally {
87         try {
88             connection.close();
89         } catch (IOException ex) {}
90     }
91 }
92 }
93
94 public static void main(String[] args) {
95     int thePort;
96     String theSite;
97     try {
98         theSite = args[0];
99         // trim trailing slash
100         if (theSite.endsWith("/")) {
101             theSite = theSite.substring(0, theSite.length() - 1);
102         }
103     } catch (RuntimeException ex) {
104         System.out.println(
105             "Usage: java Redirector http://www.newsite.com/ port");
106         return;
107     }
```

Redirector.java, HTTP重定向器, 一个简单有用的HTTP服务器

Redirector.java ✕

```
108
109     try {
110         thePort = Integer.parseInt(args[1]);
111     } catch (RuntimeException ex) {
112         thePort = 80;
113     }
114     Redirector redirector = new Redirector(theSite, thePort);
115     redirector.start();
116 }
117 }
```



功能完备的HTTP服务器

■ 流程：

- `main()` 获取主目录、端口参数
- 创建线程池 `pool`
- 每个客户端，一个线程处理
`ExecutorService.submit()`
 - 后去请求的文件路径
 - 把文件发给客户端

JHTTP.java 实现一个基本的web服务器主类
RequestProcessor.java 处理HTTP请求的线程类



JHTTP.java实现一个基本的web服务器主类

JHTTP.java

```
1 import java.io.*;
2 import java.net.*;
3 import java.util.concurrent.*;
4 import java.util.logging.*;
5 public class JHTTP {
6     private static final Logger logger = Logger.getLogger(
7         JHTTP.class.getCanonicalName());
8     private static final int NUM_THREADS = 50;
9     private static final String INDEX_FILE = "index.html";
10
11     private final File rootDirectory;
12     private final int port;
13     public JHTTP(File rootDirectory, int port) throws IOException {
14
15         if (!rootDirectory.isDirectory()) {
16             throw new IOException(rootDirectory
17                 + " does not exist as a directory");
18         }
19         this.rootDirectory = rootDirectory;
20         this.port = port;
21     }
22
23     public void start() throws IOException {
24         ExecutorService pool = Executors.newFixedThreadPool(NUM_THREADS);
25         try (ServerSocket server = new ServerSocket(port)) {
26             logger.info("Accepting connections on port " + server.getLocalPort());
27             logger.info("Document Root: " + rootDirectory);|
```

```
28
29     while (true) {
30         try {
31             Socket request = server.accept();
32             Runnable r = new RequestProcessor(
33                 rootDirectory, INDEX_FILE, request);
34             pool.submit(r);
35         } catch (IOException ex) {
36             logger.log(Level.WARNING, "Error accepting connection", ex);
37         }
38     }
39 }
40 }
41
42 public static void main(String[] args) {
43
44     // get the Document root
45     File docroot;
46     try {
47         docroot = new File(args[0]);
48     } catch (ArrayIndexOutOfBoundsException ex) {
49         System.out.println("Usage: java JHTTP docroot port");
50         return;
51     }
52
53     // set the port to listen on
54     int port;
```

JHTTP.java实现一个基本的web服务器主类

JHTTP.java

```
55     try {
56         port = Integer.parseInt(args[1]);
57         if (port < 0 || port > 65535) port = 80;
58     } catch (RuntimeException ex) {
59         port = 80;
60     }
61
62     try {
63         JHTTP webserver = new JHTTP(docroot, port);
64         webserver.start();
65     } catch (IOException ex) {
66         logger.log(Level.SEVERE, "Server could not start", ex);
67     }
68 }
69 }
```



RequestProcessor.java处理HTTP请求的线程类

RequestProcessor.java

```
1 import java.io.*;
2 import java.net.*;
3 import java.nio.file.Files;
4 import java.util.*;
5 import java.util.logging.*;
6
7 public class RequestProcessor implements Runnable {
8
9     private final static Logger logger = Logger.getLogger(
10         RequestProcessor.class.getCanonicalName());
11
12     private File rootDirectory;
13     private String indexFileName = "index.html";
14     private Socket connection;
15
16     public RequestProcessor(File rootDirectory,
17         String indexFileName, Socket connection) {
18
19         if (rootDirectory.isFile()) {
20             throw new IllegalArgumentException(
21                 "rootDirectory must be a directory, not a file");
22         }
23         try {
24             rootDirectory = rootDirectory.getCanonicalFile();
25         } catch (IOException ex) {
26         }
27         this.rootDirectory = rootDirectory;
```



```
28
29     if (indexFileName != null) this.indexFileName = indexFileName;
30     this.connection = connection;
31 }
32
33 @Override
34 public void run() {
35     // for security checks
36     String root = rootDirectory.getPath();
37     try {
38         OutputStream raw = new BufferedOutputStream(
39             connection.getOutputStream()
40         );
41         Writer out = new OutputStreamWriter(raw);
42         Reader in = new InputStreamReader(
43             new BufferedInputStream(
44                 connection.getInputStream()
45             ), "US-ASCII"
46         );
47         StringBuilder requestLine = new StringBuilder();
48         while (true) {
49             int c = in.read();
50             if (c == '\r' || c == '\n') break;
51             requestLine.append((char) c);
52         }
53
54         String get = requestLine.toString();
```

```
55
56     logger.info(connection.getRemoteSocketAddress() + " " + get);
57
58     String[] tokens = get.split("\\s+");
59     String method = tokens[0];
60     String version = "";
61     if (method.equals("GET")) {
62         String fileName = tokens[1];
63         if (fileName.endsWith("/")) fileName += indexFileName;
64         String contentType =
65             URLConnection.getFileNameMap().getContentTypeFor(fileName);
66         if (tokens.length > 2) {
67             version = tokens[2];
68         }
69
70         File theFile = new File(rootDirectory,
71             fileName.substring(1, fileName.length()));
72
73         if (theFile.canRead())
74             // Don't let clients outside the document root
75             && theFile.getCanonicalPath().startsWith(root)) {
76             byte[] theData = Files.readAllBytes(theFile.toPath());
77             if (version.startsWith("HTTP/")) { // send a MIME header
78                 sendHeader(out, "HTTP/1.0 200 OK", contentType, theData.length);
79             }
80
81             // send the file; it may be an image or other binary data
```

```
82         // so use the underlying output stream
83         // instead of the writer
84         raw.write(theData);
85         raw.flush();
86     } else { // can't find the file
87         String body = new StringBuilder("<HTML>\r\n")
88             .append("<HEAD><TITLE>File Not Found</TITLE>\r\n")
89             .append("</HEAD>\r\n")
90             .append("<BODY>")
91             .append("<H1>HTTP Error 404: File Not Found</H1>\r\n")
92             .append("</BODY></HTML>\r\n").toString();
93         if (version.startsWith("HTTP/")) { // send a MIME header
94             sendHeader(out, "HTTP/1.0 404 File Not Found",
95                 "text/html; charset=utf-8", body.length());
96         }
97         out.write(body);
98         out.flush();
99     }
100 } else { // method does not equal "GET"
101     String body = new StringBuilder("<HTML>\r\n")
102         .append("<HEAD><TITLE>Not Implemented</TITLE>\r\n")
103         .append("</HEAD>\r\n")
104         .append("<BODY>")
105         .append("<H1>HTTP Error 501: Not Implemented</H1>\r\n")
106         .append("</BODY></HTML>\r\n").toString();
107     if (version.startsWith("HTTP/")) { // send a MIME header
108         sendHeader(out, "HTTP/1.0 501 Not Implemented",
```

RequestProcessor.java处理HTTP请求的线程类

RequestProcessor.java

```
109         "text/html; charset=utf-8", body.length());
110     }
111     out.write(body);
112     out.flush();
113 }
114 } catch (IOException ex) {
115     logger.log(Level.WARNING,
116         "Error talking to " + connection.getRemoteSocketAddress(), ex);
117 } finally {
118     try {
119         connection.close();
120     }
121     catch (IOException ex) {}
122 }
123 }
124 private void sendHeader(Writer out, String responseCode,
125     String contentType, int length)
126     throws IOException {
127     out.write(responseCode + "\r\n");
128     Date now = new Date();
129     out.write("Date: " + now + "\r\n");
130     out.write("Server: JHTTP 2.0\r\n");
131     out.write("Content-length: " + length + "\r\n");
132     out.write("Content-type: " + contentType + "\r\n\r\n");
133     out.flush();
134 }
135 }
```